

# The Memory Problem

# Why LLMs "Forget"

**The Concept:** LLMs are stateless. Every prompt is treated like a brand-new interaction.

**The Challenge:** Without memory, the AI cannot handle multi-turn conversations, follow-up questions, or personal preferences.

**The "Context Window":** Like a physical desk, there is only so much "paper" (tokens) the AI can look at at one time.

# Memory Techniques

- Buffer Memory (The "Full Transcript")
  - How it works: Every single word of the conversation is saved in a list and appended to the prompt.
  - Demo behavior: It is perfectly accurate but "expensive." As the conversation grows, you send thousands of tokens just to ask a simple question.
  - Visual: Imagine a scrolling transcript that never stops growing.
- 2. Window Memory (The "Sliding Workspace")
  - How it works: We only keep the last  $K$  messages (e.g.,  $K=4$ ). When message #5 arrives, message #1 is deleted.
  - Demo behavior: This is great for speed. However, we demonstrated the "Context Cliff"—if you tell the AI your favorite color in message #1, by message #6, it will have "forgotten" it because that message fell out of the window.
  - Visual: A spotlight that only follows the most recent part of the conversation.
- Summary Memory (The "CliffNotes")
  - How it works: Instead of saving raw text, we ask the LLM to condense the past interaction into a few sentences (e.g., "The user is an AI named Gemini who likes blue").
  - Demo behavior: The AI maintains the "gist" of the conversation for a long time without using many tokens. It's efficient but might lose specific technical details.
  - Visual: A thick book being condensed into a one-page executive summary.

# Strategy 1: Buffer Memory (The "Total Recall")

- This is the simplest form of memory.
- How it works: It stores every single interaction in a list.
- Pros: The AI has perfect recall of the entire conversation.
- Cons: As the conversation grows, the "Context Window" fills up. Eventually, you hit a limit, and more importantly, it gets expensive because you pay for every token in that history for every new turn.

## Strategy 2: Window Memory (The "Short-Term Focus")

- How it works: It uses a sliding window of size  $K$ . If  $K=4$ , the AI only sees the last 4 messages.
- The Demo in run\_demo: \* With  $K=1$ , the AI forgets the user's favorite color immediately because the "New York" message pushed the "Blue" message out of the window. With  $K=10$ , it remembers easily.
- Use Case: Great for chatbots where only the immediate context matters, keeping costs predictable and low.

## Strategy 3: Summary Memory (The "CliffNotes")

- How it works: Instead of storing raw transcripts, we condense the conversation into a rolling summary.
- The Prompt: We send Current Summary + New User Input.
- Pros: It can handle incredibly long conversations without hitting token limits.
- Cons: Fine details (like a specific serial number or a middle name mentioned 50 turns ago) might get lost in the summarization process.

# Comparison

| Memory Type | Storage Method  | Cost                    | Best For...                          |
|-------------|-----------------|-------------------------|--------------------------------------|
| Buffer      | Full Transcript | High (Scales with time) | Short, high-accuracy sessions.       |
| Window      | Last K Messages | Low (Fixed)             | Customer support, quick tasks.       |
| Summary     | Compressed Text | Medium (Fixed)          | Long-form roleplay or deep research. |

Thank you!

